

Data Science in Production Exam

Spring 2026

Mads Christian Nørklit Jensen

IT University of Copenhagen
macj@itu.dk
May 29, 2026

Rules

Code of conduct

In answering the questions and working on the exercises, you are allowed to use the lecture material (slides, video recordings), as well as external resources (e.g., tutorials, books). Any use of generative AI tools (e.g., ChatGPT, Claude) is strictly forbidden, also for improving the text of your answers or the quality of the code that you wrote. Remember that this is an **individual** exam. You are not allowed to discuss the exam with other candidates. You are not allowed to share any resource, pointer, answer, code, or solution (anything, in fact) with anyone else, directly or indirectly. Not adhering to this code of conduct will result in the invalidation of the exam for all the candidates involved in the practice of information sharing.

Assignments and hand-in

You will receive a set of 10 questions (below), and a set of 8 exercises. You must correctly answer 5 questions and correctly complete 4 exercises to get a passing grade of 2. For the exercises, you will receive a directory with data; you must not modify the structure or content of that directory.

You will deliver your results in the form of a zipped directory, containing:

1. A pdf including questions and answers
2. A directory named “exercises” with one subdirectory per exercise, named “exercise- n ” (where n is the exercise number). The code and material of each exercise should be included in the exercise subdirectory, **with the exception of the data**. When running your exercises, I will assume that the data is stored in a “data” directory placed in the base directory of your assignment. You do not need to submit the data together with your assignment.

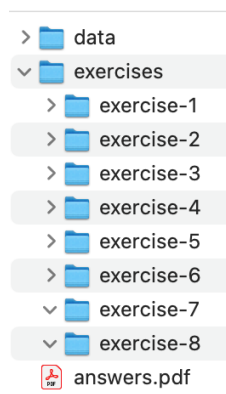


Figure 1: Example structure of your exam directory. The data directory is the one provided, but **it should NOT be included in the final hand-in** (because your examiner already has it).

Questions

Instructions.

1. Use the Latex file provided to write your answers.
2. Write your name and email in the header.
3. Write your answer right after each question.
4. Keep the question text on your answer paper, but you can erase the Rules Section in the previous page, as well as this instruction paragraph.
5. Answers should be rather brief, one short paragraph each. As a rule of thumb, all questions can be answered with 6 to 15 lines.
6. There is no need to include any formula nor illustration to answer the questions correctly. However, it is important that you *always* motivate your answers.
7. Some questions might have multiple correct answers with different motivations; you don't need to list all possible answers – one correct answer that is properly motivated is sufficient. In fact, listing many possible answers might hurt your score, as in any questions you are asked to take an operational decision to solve a problem.
8. Some questions might include information that is not relevant to solve the problem presented. This is not meant to confuse you, but rather to stimulate your reflection and reasoning. This simulates a real production environment where you are the expert in the room and other non-expert might not know the best approach to follow and thus provide information that is not relevant to the solution of the task.
9. The information needed to answer all the questions is contained in the slides and lecture recordings.

Q1. The Chief of Search of an e-commerce service is tasked to evaluate the effectiveness of the website's search functionality. To take a decision, the Chief organizes a meeting with both the Sales team and the Data Science team. The Sales team sets the criteria for what a good search result is. A good research result is the one that allows the user to spend as little time as possible parsing the list of results to find what they like, so that they can get to checkout very quickly. During the meeting, the Data Science department reports that only 1.5% of users who go to checkout end up buying two items or more, that about 70% of products are almost never purchased, and that customers who make a purchase have only a 5% likelihood to come back to the service to make another purchase within one month. After considering the requirements from Sales and the statistics from Data Science, the Chief proposes to evaluate search performance using $nDCG$. A new intern in the Data Science team suggests that $UDCG$ might represent a good improvement over $nDCG$. The intern says that, even though $UDCG$ is defined in the context of AI search, its formulation can be applied to other scenarios to flexibly account for the positional bias that the ranking algorithm might have. Do you agree that the decision taken by the Chief of Search is the best, given the criteria from Sales, and why? Do you think that the intern's suggestion is good, and why? Would you suggest to change or improve the evaluation metric, given the context presented?

A1.

- Based on the criteria from Sales, I don't agree that using $nDCG$ to evaluate search performance is best, as it gives more weight to tail relevant documents, and if the criteria is that a user has to spend as little time as possible parsing the list of results, what we care most about is very relevant results at the top of the list. I would instead suggest a combination of MAP and MRR to evaluate our search results, as MAP puts more weight on the top of our results, and this is then further honed by using MRR to put emphasis on the very first result being relevant.
- I think the intern's suggestion is bad, as $UDCG$ accounts for positional bias of LLMs, not humans which I assume our users are. $UDCG$ accounts for the fact that LLMs have different "attention spans" than humans, so if we used $UDCG$ to evaluate our search results, we would end up producing results that are catered towards LLMs rather than humans.

- Based on the fact that only 1,5% of users purchase two or more items, and that about 70% of products are almost never purchased, I would suggest several improvements to the evaluation metric.
 - First I would introduce *Coverage and Novelty* to address the 70% of items that are almost never sold. Adding these to the evaluation metric should help ensure we recommend a greater percentage of the items in our store, and ensure that we do not just only recommend the same top 3 items to users every time.
 - Second I would introduce *Diversity* to address that only 1,5% of users buy two or more items. Adding *Diversity* should ensure that users are given some variety in the type of item they are recommended (because if they come looking for 1 thing and we give them a list of results containing 20 different version of that, they will just pick one and buy that, ergo we also need variety in the items so they might put more than 1 thing in the cart).
-

Q2. A very large Web search engine uses an inverted index where postings are sorted by term frequency (TF), so that only the documents with TF higher than a given threshold are considered. The Quality Assurance Team has conducted a manual inspection of the search logs, and determined that the quality of results returned is too low. The Head of Search prepared a plan to improve retrieval the quality of the retrieved results:

- Use index elimination techniques so that low-IDF terms are not considered for indexing, thus reducing the number of potentially irrelevant documents in the set of retrieved documents.
- Index all bi-grams and tri-grams in addition to single words, so that more complex, multi-term search queries can match directly relevant documents.
- Add skip pointers to the inverted index, to skip documents that we know are not relevant. For a posting list of length m , $\frac{m}{2}$ pointers should be added.
- Provide dense retrieval functionalities by adding a new vector database that will be slowly populated over time by web crawlers. This dense retrieval functionality will be deployed with a canary deployment starting on 5% of the traffic and then slowly scaled up to phase-out the inverted index.

You are asked to review the plan before it is presented to the CTO, providing a point by point opinion. Would you change anything and why?

A2.

- Using techniques to eliminate low-IDF terms, i.e very common and non-descriptive terms such as, *and*, *the*, and so forth, is a good idea to improve the quality of the retrieved results.
- I would not suggest indexing all bi-grams and tri-grams, as this would become too space-demanding to do for all queries. I would instead suggest to only use direct n-gram indexing for very popular queries and then to implement and use a positional index for all other queries. A positional index still takes more space, but it would be more efficient than indexing all bi-grams and tri-grams, and would achieve the desired effect of more complex, multi-term search queries to match directly relevant documents.
- Adding skip pointers is a good idea to make search more efficient, especially since this is a very large Web search engine, which probably contains a lot of postings. But for a list of length m it should be $\sqrt{m}$ or $\log(m)$ skip pointers added (and they should be equally spaced). Adding $\frac{m}{2}$ skip pointers will take too much space.
- I would not recommend implementing a vector database just to phase out the inverted index. Instead I would suggest to implement the vector database, and add it to the IR system along the inverted index, so the system has both sparse and dense retrieval. The sparse retrieval through the inverted index works great for finding exact terms and phrases, whilst the dense retrieval through the vector database excel at semantic understanding, so for example handling things like synonyms

or misspellings in queries. Using canary deployment as described would be a suitable solution to deploy the new IR system that also includes a vector database, in a way that failure in the new software won't affect the majority of our users.

Q3. You're the Head of Research of a company that sells clothes online. You are sitting in a management meeting, where Marketing folks report that sales are not doing great this quarter. The website has hundreds of thousands of daily visitors, but only 3% of them press the checkout button. The CEO explains his strategy to increase revenue: nudging the few people who buy something to buy even more items at checkout. To implement this strategy, the Head of Data Science proposes to introduce a module on the checkout page that recommends related items to add to the cart. His team has already developed five recommendation system variants to suggest items related to content of the user's shopping cart. To test the relative effectiveness of the solutions, they plan it to deploy six separate canary deployments, each handling about 10% of the total unique users coming to the website daily. Five of them will implement the new recommender variants, and one will just suggest the most popular clothes, just as a baseline for comparison. The average amount of completed payment will be measured in each bucket separately, and a power analysis will be performed to establish for how long the deployments should run to appreciate an increase of 5% in the amount over the last quarter's average. The Head of Engineering argues that a bandit approach could be used instead of canary deployments, to establish the best alternative more quickly. While everyone in the room nods, you are asked to comment on the proposals of each of the three manager (the CEO, the Head of Data Science, and the Head of Engineering), and to suggest possible changes.

A3.

- While the CEO's suggestion is a valid way to increase revenue, it only has an impact on the 3%. Therefore, I would in addition to their suggestion, also suggest looking into optimizing our cold recommendations of clothes plus our recommendations when users query our website, to target the remaining 97%. Improving these should have the potential to increase the percentage of users that go to checkout.
 - To the suggestion made by the Head of Data Science, I would suggest running an A/B test first to understand the effect of implementing a new module, before we try and optimize which specific recommender variant the module should be using. If the results of the A/B test show that the module does have the desired effect, based on its statistical and practical significance, I would then proceed onward to test the different recommender variants against each other, where I would agree with the Head of Engineering and suggest a bandit approach for this.
-

Q4. The manager of a grocery store wants to run a promotional campaign with the objective of promoting "Hell's Scythe"—a specific brand of razors. The objective is to distribute 100 discounts to get as many customers as possible to buy razors from that brand. Eager to look good in the eyes of the manager and hoping to get a promotion, a member of the Data Science team rushes to propose their own solution based on frequent itemset mining. They propose to find all item pairs (X, Y) where Y represents the new razors, and that fulfill these properties: the pair should have average support, and the association rule $X \rightarrow Y$ should have high confidence and lift below 0.2. Then, customers who have been involved in past purchases that match these criteria should be offered a discount on their next purchase of "Hell's Scythe" razors. Overhearing the proposal, a colleague speaks up and notes that it does not make sense to find association rules with low lift, and proposes to modify the colleague's proposal by looking for rules with lift higher than 2. If you had to pick only one of the two proposals, which one would you pick and why?

A4. I would pick the second proposal looking for rules with lift higher than 2. This is because the lift metric is used to measure how much more often an item set occurs together than what would be expected if they are assumed to be independent. A lift score of 1 means X and Y are independent, a lift score lower than 1 indicates a negative association and a lift score above 1 indicates a positive association. If we

look for pairs with a lift of 0.2 they are less likely to be bought together, than if they were independent, whilst if we look for pairs with a lift higher than 2 means that the presence of X increases the likelihood that Y is also present.

Q5. Two entrepreneurs are working on a new startup: a book cataloging service similar to goodreads.com. In the website, books are described with metadata, and users can rate books with integer ratings from 0 to 10. Both the user base and the catalog are still very small (less than 1000 users and less than 2000 books), and the median number of ratings per user is 106. The average user rating is 6.5. The duo wants to implement a recommender system to suggest books to their users. The first founder observes that this is a perfect scenario to use exact SVD to perform recommendations: the algorithm has a high computational complexity but it is still possible to run it for such a small set of users and books, and can provide a very accurate estimate of missing ratings. The second founder argues that it would be wise to leverage the metadata associated with the books, and proposes to implement either an item-item collaborative filtering system or a two-tower approach. Comment on the pros and cons of the two proposals, motivating your answer.

A5.

- SVD fits into this scenario as it is more suited for smaller datasets as it is costly to compute, and the type of data kept in the system fits for the model. SVD does rely on existent data, meaning that for new users it will not do well, which should be considered. SVD also needs all ratings in our initial matrix to decompose it, which makes it useless for a scenario such as this where there is plenty of items with no ratings. Using SVD also does not use the metadata for the books in any meaningful way, only the ratings.
 - The second founder proposes item-item collaborative filtering to leverage the metadata, which item-item CF does not do. Using item-item CF is also more efficient when the amount of users is higher than the amount of items, which is not the case here. This approach also does not make use of the metadata of the books, it revolves around the ratings of the items, and so is reliant on their quality also. Might struggle as the rating data is quite sparse, which could lead to it being unable to find any meaningful neighbors. As the amount of items and rating increase, would also grow in cost to compute.
 - The two tower approach, would make use of the metadata of the books to build the item embeddings (the item tower), but the two tower approach would require the website to also have metadata on the users, so that another tower can build user embeddings. Using this approach is also reliant on how good the metadata is, as this would be the features used to build the towers.
-

Q6. Your sports apparel company wants to advertise its new model of running shoes online through sponsored search to increase sales. The advertising campaign will last 10 days, during which the shoes will be available at a special promotional price of 1299 DKK. The old Marketing Chief of your company has been laid off, and the new Chief still doesn't know much about online ads, so they put you in touch with the agent of an advertising marketplace. The agent offers you three different deals. The first is a cost-per-sale deal with a fee equal to the 1.9% of the shoes price, charged on each individual purchase. The second deal follows a cost-per-click model with a first-price auction, price-floored at 40 DKK per thousand clicks. The third deal follows a cost-per-mille billing model with a second-price auction, price floored at 1 DKK per thousand displays. The representative then tells you that for any of the deals you pick, you will be allowed to bid on the 30 most popular queries that the search engine receives. Alternatively, you can select a premium package to bid on a maximum of 30 queries that the marketplace will deem as the most relevant to your product. The premium package costs an extra 8% calculated on the total amount spent. Note that in the scenario of cost-per-sale you won't need to place any bid – the marketplace will display your ad alongside search results of the 30 selected queries, alternating between your ad and ads of competing advertisers in a round robin fashion. To help you with the decision, the agent tells you that past advertising campaigns of your

company achieved a CTR of 0.5% and a conversion rate of about 0.2%. You must pick *one* deal only. State which deal you pick and explain why it is better than the other two deals you discarded.

A6.

Based on our CTR of 0.5% and conversion rate of 0.2% in past advertising campaigns, we can estimate the amount of people that need to see the ad for one of them to click it, as well as how many people we need to click the ad before we achieve a sale: $\frac{1}{0.005} =$ we need 200 impressions for one click. $\frac{1}{0.002} =$ we need 500 clicks to achieve a sale. With this information we can estimate the prices of each deal:

- Cost-per-sale: $1299 * 0.019 = 24.618$ (round up to 25 dkk per sale).
- Cost-per-click: Minimum price achievable is 40 dkk per thousand clicks, which would mean a price per click of $\frac{40}{1000} = 0.04$ dkk per click, and our past advertising campaigns suggest we need 500 clicks for a sale, meaning we would be paying $500 * 0.04 = 20$ dkk per sale (if we win the auction at floor price).
- Cost-per-mille: Lowest price achievable would be 1 dkk per 1000 seeing the ad. We achieve a click on the add per 200 impressions, and we need 500 clicks to achieve a sale, meaning $200 * 500 = 100000$ impressions for a sale. This results in us paying $\frac{100000}{1000} = 100$ dkk per sale (if we win the auction at floor price).

Based on the information I would pick the cost-per-sale deal with the premium package. While the cost-per-click deal has the potential to be lower cost, it also carries the risk of being much more expensive than the best case scenario if the bids of the queries are above the floor price (which I would expect that there is a substantial chance they will be). The CPS deal should therefore in most scenarios bring us the biggest profit per sale. I pick the premium package because an 8% increase in cost per sale for the chosen model $25 * 0.08 = 2$ dkk, is not a lot compared to having better queries that could increase our CTR and conversion rate.

Q7. A new music recommendation service has implemented a new system of catalog creation that can index songs from many sources. The service ingests thousands of new songs every day, and makes them available to its users. Since the songs are taken from many different providers, song metadata are sparse and with varying formats. The service is still struggling to attract users, and at the moment only a very small set of users is using it. Typically, these users get recommendations and rate the songs they have been recommended after they listen to them. Existing users are extremely active though, as they listen to many songs and often rate them. The management suspects that the site is not doing so well because the recommendations are given at random (i.e., randomly selected songs are recommended), and wonders which recommendation strategy should be implemented to increase the engagement of current users, and to provide meaningful recommendations to the new users who land on the platform for the first time. What solution would you recommend to implement and why?

A7. Since we have a system with different circumstances, in this case both established users and items, as well as cold starts for these, I would suggest a solution of a hybrid recommender approach, where the approach taken is switching, so we adapt the recommender used to the specific circumstance. I'd suggest a content-based approach for new songs ingested into the system, a popularity-based approach for brand new users, and finally a multi-armed bandits approach for recommendations for our established users. Choosing bandits is also motivated by the fact that the service ingests thousands of new songs each day, and bandits are well suited to handle a dynamic catalog like this.

Q8. Your company is at a historical crossroad. The CEO has decided to rebrand the logo color. The current logo is blue, and the design department has shortlisted magenta, orange, and brown as new color candidates. The Data Science department is tasked with identifying which alternative best drives user engagement. After some deliberation, the team identifies 10 engagement metrics and designs an experiment. They propose to split live traffic into four buckets of equal size: one control bucket displaying the logo with a

neutral color (black), and three treatment buckets showing each candidate color. For each treatment bucket, a T-test will be run per metric continuously as data accumulates, and each bucket will be evaluated as soon as each of its 10 T-tests independently crosses $p < 0.05$. Whichever treatment has the most metrics with a statistically significant improvement over control will be recommended to the CEO. To ensure the experiment is rigorous, the team also plans to run a power analysis after the experiment concludes, to verify that the sample sizes were sufficient to detect a 5% relative change in each metric. Comment on the value of the Data Science Department plan, and propose changes if needed.

A8. The general plan of running A/B tests to determine which, if any, logo color would provide the greatest increase to user engagement is a good approach when making a long-lasting design choice such as this. I would suggest several modifications of the outlined plan though.

First, in terms of the metrics having 10 metrics that are treated equally is an issue. I'd propose finding 1 main driver metric, and a maximum number of metrics we measure being 5. Furthermore I would also request identifying guardrail metrics so we have something that measures undesired outcomes, that the logo change might introduce. If we have several metrics we need to measure I would suggest using a Bonferroni correction.

The second change I'd propose would be to run a power analysis to determine the sample sizes for control and treatment, instead of the four bucket of equal sizes split, and I'd also suggest using the current logo as the control rather than a new color as suggested.

Third thing I'd change is determining how long the experiment should run, instead of doing P-value peeking and letting the experiment length be dependent on when $p < 0.05$. Results fluctuate and can appear significant early on, but the experiment should be run at the bare minimum as long as it was designed for, and running it longer is better.

Finally, when the results are collected, their statistical significance should then be measured with T-tests, and we should also measure the practical significance of them as well. After this it should be clear which treatment provided the desired effect/value and it should be recommended to the CEO.

Q9. The director of a very popular online political magazine has embraced the idea of adding a recommender system to suggest personalized editorials to readers. Every day, 5 to 10 candidate editorials are available, and the recommender system needs to pick one per reader, to be presented on their personalized front page. The magazine is paywalled, and only users with an account linked to a subscription can access it. The Chief of Operations suggests to implement a EXP3 algorithm to provide the recommendations. They propose to map each editorial to a semantic embedding space using state-of-the-art embeddings. The profile of each user can be then calculated based on the average embedding of the editorials that the user has read in the past. They plan to rollout a separate personalized model for each user, but considering only users who have read at least 30 editorials, to avoid profile sparsity. The remaining users will be recommended the most popular editorials. The reward of the EXP3 models is equal to 1 if the user reads the recommended editorial within the first 30 minutes of it being displayed, and zero otherwise. As the recommender system is rolled out, all the interaction logs will be collected and used offline to develop and evaluate better recommenders using the Replay technique. What is your opinion about the Chief of Operations plan? Would you change anything and why?

A9. While I agree with the direction the Chief of Operations is going, with using a multi armed bandits approach wherein each arm represents a candidate editorial, there are things I would propose to change. Instead of using the EXP3 algorithm, which is a context-free bandits approach, I would suggest implementing Linear UCB which is a contextual bandits algorithm. Using Linear UCB will allow us to use the feature vectors of the editorials and users. I support the strategy to only consider users with a backlog of read editorials, so there are features to extract, and I agree with using a popularity filter to recommend to cold users, but I disagree with a separate personalized model for each user. It should be one model for all users, that is continuously updated based on the features and rewards. Finally collecting all interaction logs, and using them for offline evaluation with the Replay technique is the most suitable, as it is a much more risk free way than to do online evaluation on the live system.

Q10. Your manager has just seen a Youtube tutorial about Learning To Rank (LTR)

and they are so excited about it that they want you to implement it for the company's Web search engine. The manager proposes to implement the following workflow. For each query, LTR should be used first to rank all possible results. Then, to improve the quality of top 100 results, a re-ranking based on the documents PageRank should be performed, to focus only on authoritative sources. Finally, the top 10 results should be re-ranked using the BM-25 score calculated against the query to maximize relevance. The manager plans to hire workers to annotate data for training the LTR model. Each entry in the dataset will contain a (user, query, url) tuple taken from the last month of user traffic. Each entry signifies that a user clicked an url right after submitting a query. The dataset will be split at random between training and test. The training set will be annotated by the workers with a ternary label (non-relevant, partly relevant, relevant). Do you agree with the manager and why? What improvements would you possibly propose, if any?

A10. While I can agree to the overall plan of implementing LTR and using model chaining, there are several changes I would suggest. First and foremost LTR should run on all possible results as it is inefficient, so for the native ranking for each query I would suggest using BM25. Then we can proceed to improve the quality of the top 100 results with LTR, PageRank should not be the sole feature used for this ranking, but it works as part of what is fed into the LTR algorithm. Then I'd suggest running LTR optimizing with nDCG for the top 50 results, and finally LTR optimizing for MRR for the top 10 results, to try and maximize the relevance of the very first result returned from our search engine. Hiring workers to annotate data so the model has relevance judgements for its training is very good, but should rather be used to label (query, doc) pairs, wherein they judge the relevance of top k documents. Things such as user clicks is a separate implicit relevance judgement, that should also be included in the training. Finally we should choose a ratio between training and test data instead of splitting it at random. The randomization should just be shuffling the data around before doing the split. I'd suggest 2/3 for training and 1/3 for testing.

Exercises

Instructions.

1. Follow the directory structure illustrated in the Rules section (first page of this pdf).
 2. Use Python notebooks whenever possible.
 3. When handing in your assignment, the Python notebooks **should contain the output of executed cells**.
 4. When design or implementation details are not specified, you can make your own choices.
-

Exercise 1: Indexing

Use Elasticsearch to index and query the grocery dataset (directory data/grocery). The dataset contains a catalogue of items sold in a store. Indications on how to perform the indexing and querying are provided next. Write your code in a single Python notebook. You are free to use the Elasticsearch wrapper API, the custom helpers we have used in class, or both. For each of the points below, write some brief comments (or text cells) in your notebook to describe how you addressed that specific point. Use the numbering provided below to refer to the points in your notebook (e.g., “*Point #1: I took care of efficient loading by ...*”)

Indexing. Please index the dataset taking into account the following points:

1. The dataset needs to be loaded efficiently.
2. This is intended to be a rather dynamic database, where updates done multiple times per hour
3. Create an index with 2 replicas.
4. The documents should allow full text search on the item name and category name, and should allow filtering and sorting by category and price.
5. A relevance metric that allows for text length discounting should be used. Increase by 30% the amount of length discounting compared to the default value.

Querying. For each of the points below, write a separate Python parametric function that performs:

6. An exact-match query on the item name.
7. A full-text query on item name and category, with category boosted by a factor 4 compared to item name.
8. A full-text query on the category that sort the results by price (lowest to highest).
9. A fuzzy query on the item name.

Test your functions with at least one example query each.

Exercise 2: Relevance feedback

Use Elasticsearch to index the Book Plots dataset (directory data/books). It contains the fields: (WikiID, Title, Author, Date, Summary). Write a Python function that implements pseudo-relevance feedback in Elasticsearch. Specifically, when a full-text query is submitted, a set of relevant documents is retrieved, based on a match on the Summary field. Out of this set, important terms should be identified and used to expand the query. The expanded query is then submitted to the index and the final result returned.

Write your code in a single Python notebook. You are free to use the Elasticsearch wrapper API, the custom helpers we have used in class, or both. You should write your own code to implement the relevance feedback functionality; do not use additional plugins or libraries.

Test your function with the queries **alien ship**, **dream**, and **zombie**. Compare the results with full-text queries that do not use expansion.

Exercise 3: Learning to rank

The Microsoft Learning To Rank Challenge dataset contains features extracted from (query,url) pairs along with relevance judgements. The queries, urls and features descriptions are not given, only the feature values are. The data is available in the directory (data/microsoft_ltr), and contains a training and test sets, see readme file for format details (note that the files are in RankLib format with sparse representation of features, which may require the use of the -sparse option for loading).

Train a set of LTR models (ListNet, RankNet, and LambdaMART) and evaluate their performance on the test set using MRR@10, NDCG@10, and P@10. Summarize the results in three barplots, one for each evaluation metric, where each bar represents the performance of one of the LTR models. Write all your code in a Jupyter notebook. In a text cell, comment briefly the results: Did you expect specific differences between models? Any remark about the results obtained across performance metrics?

Exercise 4: Frequent itemsets

To learn more about their customers' behavior, the Tesco marketing department is interested in three pieces of information.

- A list of triplets of products (A, B, C) such that C is purchased frequently after A and B . They are interested in strong associations, such that the likelihood of the association is at least 1.8 times higher than chance.
- A list of pairs (A, B) such that B is frequently purchased after A , and that the pair (A, B) is found least in 0.5% of the carts.
- The top three items by confidence in the association rule $A \rightarrow B$, such that A is "Kettle Chips Sea Salt And Black Pepper Corns 150G," and the rule has 1.05 lift.

They provide you with a list of carts (file data/tesco/tesco_carts_clean.csv). Each cart is a list of product IDs. The respective descriptions of those ids is given in the file data/tesco/tesco_inventory_clean.csv. Write a Python notebook that satisfies the request of the marketing department. Present the association rules you find using the *names* of items, and not their IDs. Explain your choices briefly in text cells of your notebook.

Exercise 5: Recommender system

You are provided with a dataset of restaurant ratings from google (data/restaurants). The dataset contains reviews in the form (user_id, business_id, rating, review_text, test), where test is set to 0 for the training set and to 1 for the test set.

Implement two recommender systems trained on the training set and tested to predict the ratings in the test set:

1. Item-Item collaborative filtering
2. SVD

Evaluate the results using Root Mean Square Error (RMSE) calculated between the predicted rating and the actual rating.

Then, implement a third, content-based recommender. Calculate the feature vector of a restaurant as the 5 top TF-IDF words appearing in the reviews of that restaurants in the training set. Then, produce a ranked list of the top-5 recommended restaurants for user 116396021024611789723 (i.e., the user with the highest number of ratings in the training set). List restaurant IDs and their feature vectors.

Write all your code in a single Python notebook, and present the results in two bar plots, one for each of the two metrics, where each bar reports the score of one of the methods. To implement the recommender systems, you can use custom code or rely on existing Python libraries, or both.

In a text cell in the notebook, write a brief reflection about the results obtained, highlighting expected findings and surprising findings, if any.

Exercise 6: Bandits!

You are provided with a list of ratings that users expressed after consuming news items. The data consists of just two columns, one for the news article type (a number from 0 to 5) and one for the rating (a 'reward' in the range $[0, 10]$). The user ids are not reported – you can pretend that all the rewards come from the same user. The data is in the directory `data/bandits`.

Find a valid way to evaluate two bandits algorithms of your choice in an offline fashion on the provided dataset. As evaluation metric, you should use cumulative reward after 1,000 arm pulls (total). Compare the performance of the two algorithms with one naive strategy that explores always a random arm (full exploration). For each algorithm (including the baseline) plot how the cumulative reward grows over time (using one or more lineplots).

Write all your code in a Python notebook, and in a text cell write a brief comments about the result, mentioning especially expected and unexpected findings.

Exercise 7: A/B testing

A company that sells online videogames has implemented a new recommender system and wants to try it out. The hope is that the recommender system will increase the main driver metric of the company, which is the average money spent per purchase. Before rolling the system out, the Chief Operating Officer of the company decides to run an A/B test, where 20% of the traffic is directed to the new AI system. The service sells 500 copies of digital games every day. After seven days, the manager stops the A/B test and hands the data over to the Data Science department (you), to establish whether the new AI system had an effect. Management would like to see an *absolute* increase of 0.50 EUR in their driver metric.

In the directory (`data/abtest`) you find a table with two columns: the first including the money spent in a videogame purchase, the second containing a marker that indicates whether that rating was collected prior to the A/B test (*historical*), or during the A/B test in the *control* group or in the *treatment* group (i.e., those customers exposed to the AI service).

Write some Python code in a notebook to analyze the data, determine the significance and magnitude of the effect, and provide a verdict to the Chief Operating Officer. Write also some code to check whether the A/B test ran for long enough; if not, provide a recommendation on how much longer the test should run for. Comment your reflections and coding choices in the code, using text cells.

Exercise 8: DevOps (double points)

Deploy a recommender system on Kubernetes. This system is composed by 5 microservices:

1. A user activity simulator that generates (user, item) pairs from MovieLens. The simulator can be started and stopped by you using a web interface [code partly provided]
2. A recommender module that given a (user, item) pair in input, returns an estimated score for it, using a pre-trained model. You should train the model based on the movielens data provided (directory `data/movielens`) and you can use any model of your choice.
3. A feedback collector module that given a (user, item, actual_score, estimated_score) tuple, keeps track in memory of i) the average error (RMSE) of the estimated vs actual score, ii) the unique number of users who provided a score.
4. A Prometheus monitoring service that collects RMSE and number of unique users from the feedback collector module
5. A Grafana service that visualizes the metrics collected by Prometheus on a dashboard.

Deliver a folder with all configuration file and resources (e.g., Python files, Docker files) that are needed to build the project. In the same directory, provide a screenshot of your Grafana dashboard after a run of 3 minutes. The backend microservices must be implemented in Python. You must use the `user_simulator.py` file provided (`data/devops`) to implement the activity simulator. You can use any tool you like to implement the frontend. Use Linux-based images.